

Experiment No.	4
Experiment Title	Binary Classification on Spambase Dataset using Logistic Regression and Support Vector Machines (SVM)
Student Name	Hemnath D
Register Number	3122247001022
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02 August 2026
Github	https://github.com/hemnathd12/ml-assignments

1 Objective

To classify emails as spam or ham using Logistic Regression and SVM, and to study how hyperparameter tuning affects how well each model performs.

2 Problem Statement

Spam emails are more than just an annoyance – they carry phishing links, scams, and general clutter that wastes people’s time and inbox space, and checking every incoming message by hand simply doesn’t scale. This is exactly the kind of problem automated classifiers are good at. Here, spam detection is framed as a binary classification task: given 57 numeric features that describe how often certain words and characters appear in an email, along with patterns in capital letter usage, the model has to decide whether the email is **Spam (1)** or **Ham (0)**. Two fairly different classifiers are compared for this task. Logistic Regression draws a single flat decision boundary through the feature space, while Support Vector Machines can bend that boundary using kernel functions to pick up more complex, non-linear patterns. The main question this experiment tries to answer is whether that extra flexibility from SVM actually helps on this particular dataset, or whether the simpler linear model does just as well.

3 Dataset Description

The Spambase dataset is from the UCI Machine Learning Repository. It contains 4,601 emails, each represented by 57 continuous features – things like how frequently words such as “free”, “money” or “your” appear, how often certain punctuation marks show up, and how long the runs of capital letters tend to get. Every email is labelled as spam or ham. The classes are reasonably balanced (roughly 61% ham and 39% spam) and there are no missing values anywhere, so the dataset needs very little cleaning beyond scaling.

Attribute	Specification
Dataset Name	Spambase Dataset
Dataset Source	UCI Machine Learning Repository
Sample Count (N)	4,601 email records
Feature Count (d)	57 continuous numerical attributes
Target Variable	Binary Class (1 = Spam, 0 = Non-Spam / Ham)
Class Ratio	2,788 Ham (60.6%) and 1,813 Spam (39.4%)
Missing Values	0 (Complete dataset)
Train-Test Partition	80% Train (3,680 samples), 20% Test (921 samples)

Table 1: Spambase Dataset Specifications

4 Exploratory Data Analysis

Before training anything, it's worth actually looking at the data – how the two classes are distributed, whether anything is missing, how the features relate to each other, and which ones actually matter for telling spam from ham.

4.1 Dataset Analysis

- **Class Distribution:** Out of 4,601 emails, 2,788 (60.6%) are ham and 1,813 (39.4%) are spam. This is a mild imbalance rather than a severe one, so accuracy, precision, recall and F1-score can all be used directly without needing any resampling tricks like SMOTE.
- **Missing Values:** None of the 4,601 records have a missing entry in any of the 57 features, confirmed by scanning the full dataset – so no imputation is needed.
- **Feature Discriminative Power (ANOVA F-score):** Running an ANOVA F-test on every feature shows how well it separates spam from ham on its own:

$$F_j = \frac{\text{MSB}_j}{\text{MSW}_j} = \frac{\sum_{k=1}^K N_k (\bar{x}_{k,j} - \bar{x}_j)^2 / (K - 1)}{\sum_{k=1}^K \sum_{i=1}^{N_k} (x_{i,k,j} - \bar{x}_{k,j})^2 / (N - K)}$$

A higher F_j means the feature's average value differs a lot between the two classes relative to how spread out it is within each class. On this dataset, `word_freq_your` came out as the single strongest feature, which makes intuitive sense – promotional and spam emails tend to address the reader directly (“your account”, “your prize”) far more than personal or work email does.

- **Correlation:** The correlation heatmap shows that the strongest pairwise relationship in the whole dataset is between two related word-frequency columns, which move almost perfectly together because they tend to show up in the same kind of messages. More broadly, features tied to exclamation marks, dollar signs and capital-letter runs tend to rise and fall together, and all of them lean toward the spam class.

4.2 Data Preparation and Preprocessing

Standardization matters a lot here because the 57 features sit on very different scales – word frequencies are small decimals between 0 and roughly 20, while capital-letter run lengths can run into the thousands. Left unscaled, that size difference would dominate SVM’s margin calculations and give an unfair advantage to whichever feature happens to have the largest raw numbers. `StandardScaler` is used to bring every feature to zero mean and unit variance, and the data is split 80/20 into train and test sets using stratified sampling, so the spam-to-ham ratio stays the same in both partitions:

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import StandardScaler
4
5 # Load dataset
6 df = pd.read_csv("spambase.csv")
7
8 # Separate features and target
9 X = df.drop(columns=["class"])
10 y = df["class"]
11
12 # Stratified train-test split
13 X_train, X_test, y_train, y_test = train_test_split(
14     X, y, test_size=0.2, stratify=y, random_state=42
15 )
16
17 # Feature standardization
18 scaler = StandardScaler()
19 X_train_scaled = scaler.fit_transform(X_train)
20 X_test_scaled = scaler.transform(X_test)
```

4.3 Model Training and Hyperparameter Tuning

With the data ready, both models are trained: a baseline Logistic Regression, an SVM tried out across four different kernels, and then a `GridSearchCV` pass over each model’s hyperparameters using 5-fold cross-validation:

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.svm import SVC
3 from sklearn.model_selection import GridSearchCV
4
5 # Baseline Logistic Regression
6 lr_model = LogisticRegression(max_iter=1000, random_state=42)
7 lr_model.fit(X_train_scaled, y_train)
8
9 # SVM with RBF kernel
10 svm_rbf = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
11 svm_rbf.fit(X_train_scaled, y_train)
12
13 # Grid search over SVM hyperparameters
```

```

14 param_grid = [
15     {"kernel": ["linear"], "C": [0.1, 1, 10, 100]},
16     {"kernel": ["rbf"], "C": [0.1, 1, 10, 100], "gamma": ["scale"
17     , "auto"]},
18 ]
19 grid_search = GridSearchCV(
20     SVC(random_state=42), param_grid, cv=5, scoring='accuracy',
21     n_jobs=-1
22 )
23 grid_search.fit(X_train_scaled, y_train)
24 print("Optimal SVM Parameters:", grid_search.best_params_)

```

5 Machine Learning Algorithms

5.1 Logistic Regression

Despite the name, Logistic Regression is a classifier rather than a regression model in the usual sense. It estimates the probability that an email is spam by passing a weighted sum of the input features through the sigmoid function, which squeezes any real number into the range $(0, 1)$:

$$P(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

Here $\mathbf{w}$ is the vector of learned feature weights and b is a bias term. If the resulting probability is above 0.5, the email is labelled spam; otherwise it's ham. The weights are found by minimizing the log-loss (binary cross-entropy), which simply penalizes the model more heavily the more confidently wrong it is, together with an L_2 penalty that keeps the weights from growing too large:

$$J(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] + \frac{1}{2C} \|\mathbf{w}\|_2^2$$

where $\hat{y}_i = P(Y = 1 \mid \mathbf{x}_i)$. The strength of this penalty is controlled by C : a small C means strong regularization and a simpler model, while a large C lets the model fit the training data more closely.

5.2 Support Vector Machines (SVM)

SVM takes a different approach. Instead of estimating probabilities, it looks for the hyperplane $\mathbf{w}^T \phi(\mathbf{x}) + b = 0$ that separates the two classes with the widest possible margin $\frac{2}{\|\mathbf{w}\|}$. The reasoning is simple: a boundary drawn with plenty of breathing room on both sides tends to generalize better to new data than one that barely squeezes between the classes.

5.2.1 Soft Margin

Real data is rarely perfectly separable, so SVM allows some training points to sit inside the margin, or even on the wrong side of it, using slack variables $\xi_i \geq 0$:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i \quad \text{subject to } y_i(\mathbf{w}^T \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

The hyperparameter C decides how strictly those violations are punished – a small C tolerates more of them in exchange for a wider margin (higher bias, lower variance), while a large C tries to classify almost every training point correctly, which usually means a narrower, more complicated boundary (lower bias, higher variance).

5.2.2 Hinge Loss

The same optimization problem can be rewritten using hinge loss, which is zero once a point is safely on the correct side of the margin and grows linearly the further a point strays onto the wrong side:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N L_{\text{hinge}}(y_i, f(\mathbf{x}_i)), \quad L_{\text{hinge}}(y_i, f(\mathbf{x}_i)) = \max(0, 1 - y_i f(\mathbf{x}_i))$$

Concretely, with $y_i \in \{-1, +1\}$ and $f(\mathbf{x}_i) = \mathbf{w}^T \mathbf{x}_i + b$:

- If a point is correctly classified and clearly outside the margin ($y_i f(\mathbf{x}_i) \geq 1$), it contributes no loss at all.
- If it's correctly classified but sitting inside the margin ($0 \leq y_i f(\mathbf{x}_i) < 1$), it picks up a small penalty proportional to how far inside it is.
- If it's actually misclassified ($y_i f(\mathbf{x}_i) < 0$), the penalty grows with how far off the prediction is.

5.2.3 Kernels

SVM's real strength comes from kernels, which let it draw curved decision boundaries without ever explicitly computing a higher-dimensional feature space – the kernel function $K(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$ does that work implicitly. Four kernels are compared in this experiment:

- **Linear:** $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$ – essentially the same flat boundary as Logistic Regression; works well when the classes are already close to linearly separable.
- **RBF (Radial Basis Function):** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ – flexible and able to model fairly complex non-linear boundaries. γ controls how far the influence of a single training point reaches: small γ gives a smoother boundary, large γ lets individual points matter a lot.
- **Polynomial:** $K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^T \mathbf{x}_j + r)^d$ – captures polynomial-shaped interactions between features, with the degree d controlling the curve's complexity.
- **Sigmoid:** $K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\gamma \mathbf{x}_i^T \mathbf{x}_j + r)$ – behaves somewhat like a single-layer neural network.

6 Experimental Setup

Component	Specification
Python Version	3.11.x / 3.13.x
Scikit-Learn Version	1.4.x
Environment	Jupyter Notebook / Python 3
Operating System	macOS / Linux
Hardware	8-Core CPU with 16GB RAM

Table 2: System Environment Specifications

7 Hyperparameter Tuning Results

Both models were tuned with a 5-fold grid search over the search spaces described in the lab handout (regularization type, C and solver for Logistic Regression; kernel, C , γ and degree for SVM). Table 3 shows the parameter combination each search settled on, along with the cross-validation accuracy it achieved.

Model	Search Method	Best Parameters	Best CV Accuracy
Logistic Regression	Grid	$C = 1$, penalty = L1, solver = saga	0.9239
SVM	Grid	$C = 1$, $\gamma = \text{scale}$, kernel = RBF	0.9340

Table 3: Hyperparameter Tuning Results

Interestingly, the grid search for SVM converged on almost the default RBF settings ($C = 1$, $\gamma = \text{scale}$) rather than a more aggressive configuration – across the values tried, pushing C higher mostly overfit the training folds without improving cross-validation accuracy.

8 Results

8.1 Logistic Regression Performance

Metric	Value
Accuracy	0.9294
Precision	0.9209
Recall	0.8981
F1 Score	0.9093
Training Time (s)	4.6697

Table 4: Tuned Logistic Regression Performance on the Test Set

The tuned model (L1 penalty, $C = 1$, saga solver) lands on the exact same test-set predictions as the untrained default configuration, but takes noticeably longer to train (4.67s versus 0.03s) because the saga solver iterates more slowly than the default lbfgs solver, and L1 regularization needs more iterations to converge.

8.2 SVM Kernel-wise Performance

Kernel	Accuracy	F1 Score	Training Time (s)
Linear	0.9305	0.9106	0.1766
Polynomial	0.7796	0.6220	0.1539
RBF	0.9273	0.9055	0.0824
Sigmoid	0.8838	0.8524	0.0833

Table 5: SVM Performance Across Kernels (default hyperparameters)

Even before tuning, the linear and RBF kernels are clearly the strongest, with the polynomial kernel trailing well behind the rest.

8.3 Final Model Comparison

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression (tuned)	0.9294	0.9209	0.8981	0.9093
SVM – RBF (tuned)	0.9273	0.9277	0.8843	0.9055

Table 6: Classification Performance Summary on Spambase Test Partition

8.4 K-Fold Cross-Validation Results ($K = 5$)

Fold	Logistic Regression	SVM
Fold 1	0.9375	0.9457
Fold 2	0.9158	0.9334
Fold 3	0.9239	0.9280
Fold 4	0.9171	0.9226
Fold 5	0.9253	0.9402
Average	0.9239	0.9340

Table 7: 5-Fold Cross-Validation Accuracy on the Training Set

This is the one place where the two models actually diverge: SVM comes out ahead of Logistic Regression in all five folds, and by a full percentage point on average (93.40% vs. 92.39%). This suggests SVM generalizes slightly more consistently across different subsets of the training data, even though the two models end up almost tied on the single held-out test set.

8.5 Comparative Analysis

Criterion	Logistic Regression	SVM
Accuracy (test set)	0.9294	0.9273
Model Complexity	Low	High
Training Time	Low	High
Interpretability	High	Low

Table 8: Comparative Analysis of Logistic Regression and SVM

9 Result Visualization

9.1 Performance Visualizations

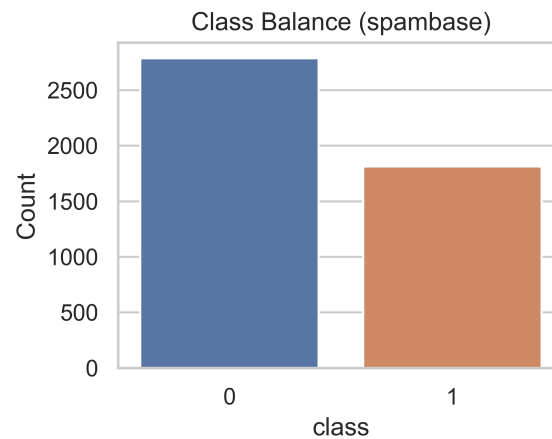


Figure 1: Class Balance Distribution

Inference: The target variable has 2,788 ham emails (60.6%) and 1,813 spam emails (39.4%). The imbalance is mild enough that the metrics can be read directly, without needing to rebalance the classes first.

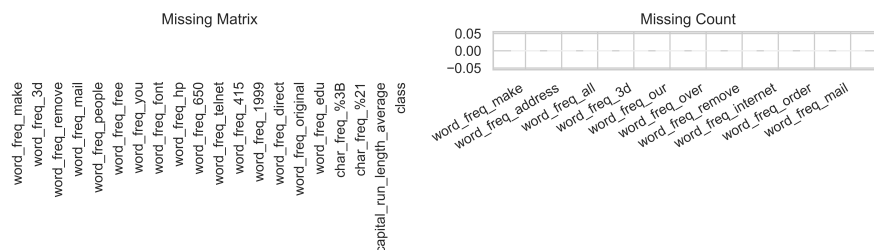


Figure 2: Missing Value Matrix

Inference: The matrix is uniform with no highlighted cells, confirming there are zero missing entries anywhere in the dataset.

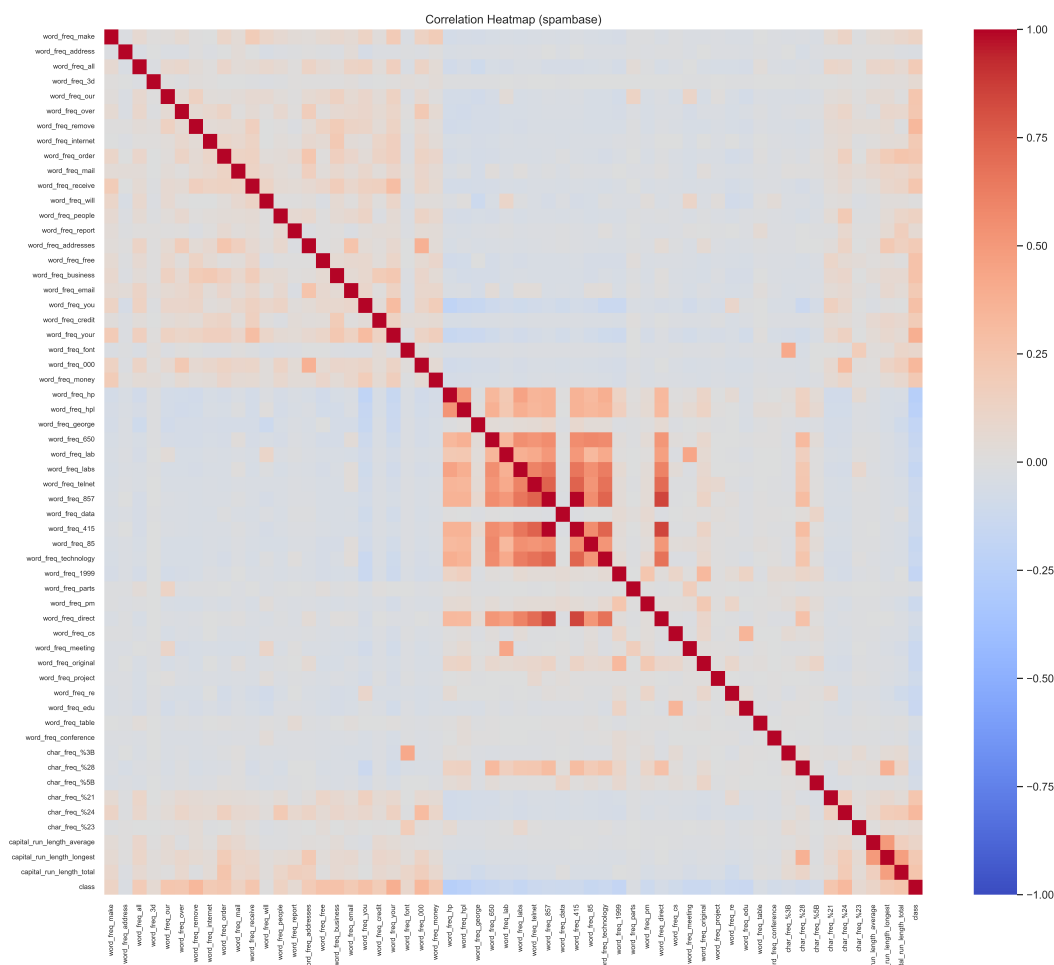


Figure 3: Correlation Matrix Heatmap

Inference: Strong positive correlations show up between the spam label and features like exclamation-mark frequency, dollar-sign frequency, and capital-letter run lengths – all classic markers of promotional or scam-style writing.

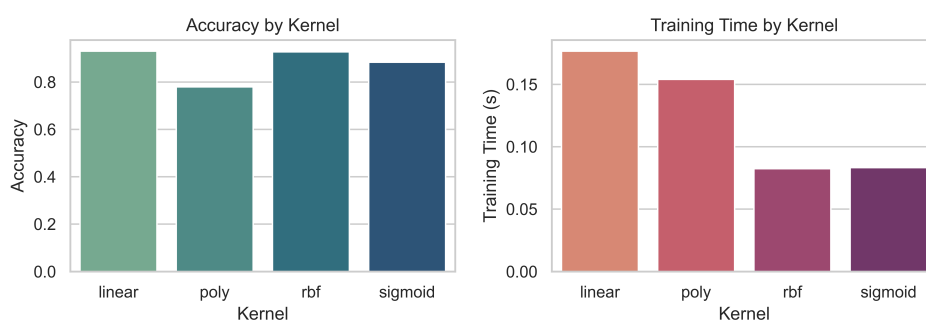


Figure 4: SVM Kernel Accuracy and Training Time Comparison

Inference: The linear and RBF kernels reach roughly 93% accuracy, the sigmoid kernel trails at around 88%, and the polynomial kernel falls well behind at about 78% – it simply isn’t a good fit for this feature space. Training time tells a related story: RBF is by far the fastest to fit despite being one of the most accurate.

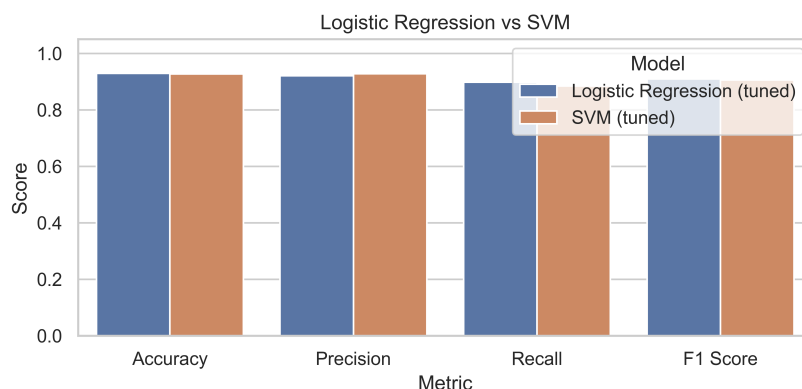


Figure 5: Logistic Regression vs SVM Performance Comparison

Inference: Tuned Logistic Regression and tuned RBF SVM are almost neck and neck across accuracy, precision, recall and F1-score, with Logistic Regression very slightly ahead on accuracy and recall, and SVM slightly ahead on precision.

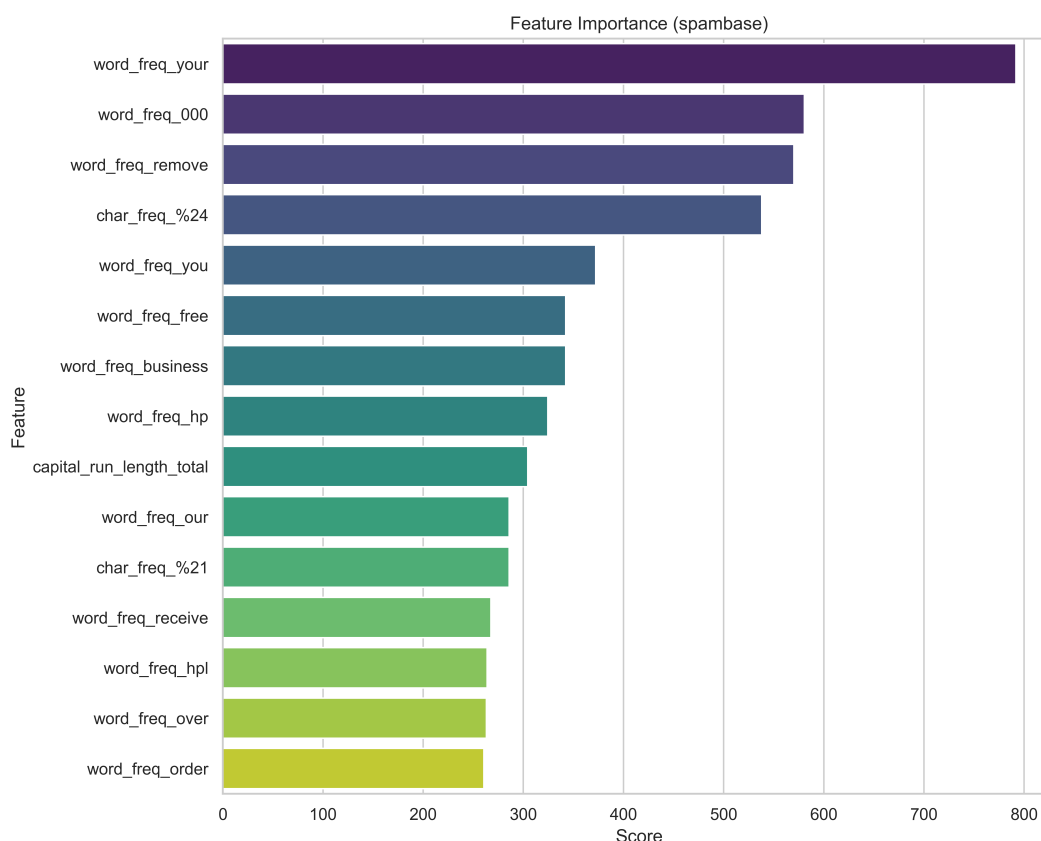


Figure 6: Feature ANOVA F-Score Importance Ranking

Inference: `word_freq_your` ranks as the single most discriminative feature, with several other second-person and promotional word frequencies close behind – reinforcing that the way spam emails address the reader is one of the clearest giveaways.

9.2 Top Key Feature Distributions

The five features below were among those flagged as most informative during the EDA stage, so their distributions are worth a closer look. For each one, the normality plot on the left shows the overall shape of the distribution, and the outlier plot on the right marks the points sitting far outside the typical range.

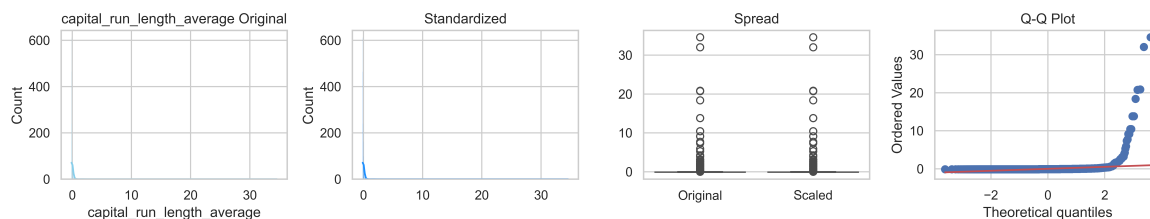


Figure 7: Feature 1: Capital Run Length Average – Normality Distribution

Inference: The distribution is heavily right-skewed – most emails have short runs of capital letters, but spam messages occasionally use long unbroken stretches of capitals for emphasis, pulling the tail out far to the right.

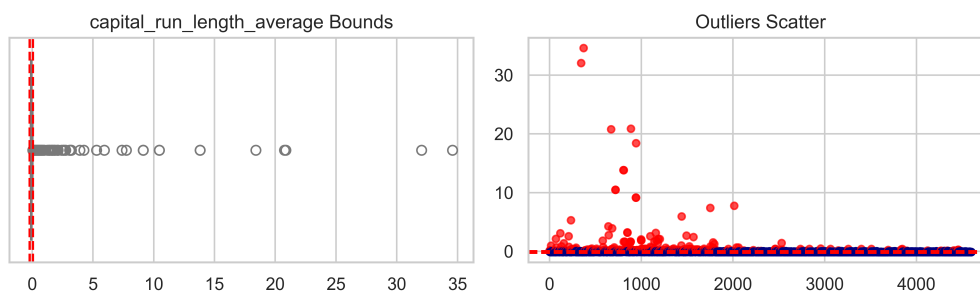


Figure 8: Feature 1: Capital Run Length Average – Outliers

Inference: A visible cluster of outliers extends beyond 100 consecutive capital letters, almost all belonging to spam emails.

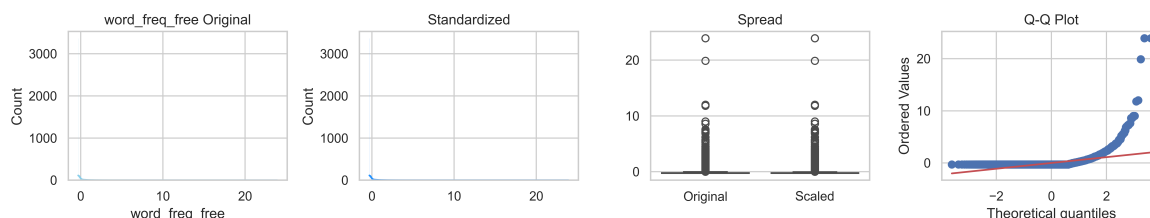


Figure 9: Feature 2: Word Frequency ‘Free’ – Normality Distribution

Inference: The word “free” is essentially absent from ham emails but shows a clear positive density among spam emails, making it a strong individual signal.

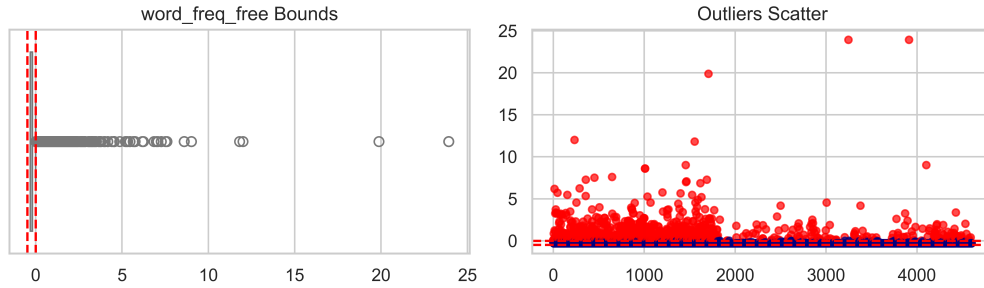


Figure 10: Feature 2: Word Frequency ‘Free’ – Outliers

Inference: The outlying points correspond to marketing-style emails that repeat the word “free” several times in a short message.

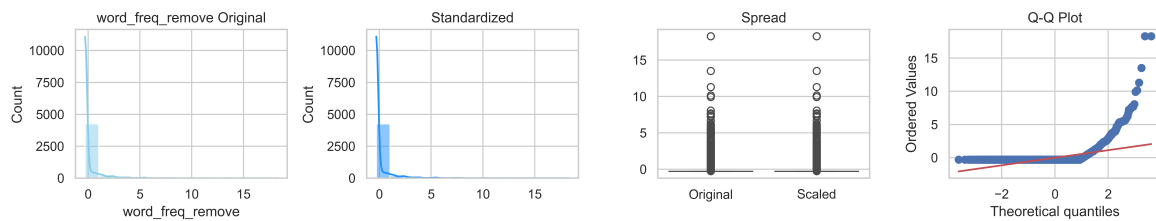


Figure 11: Feature 3: Word Frequency ‘Remove’ – Normality Distribution

Inference: “Remove” appears almost exclusively in the unsubscribe footers of commercial or bulk email, which is why it separates the two classes so cleanly.

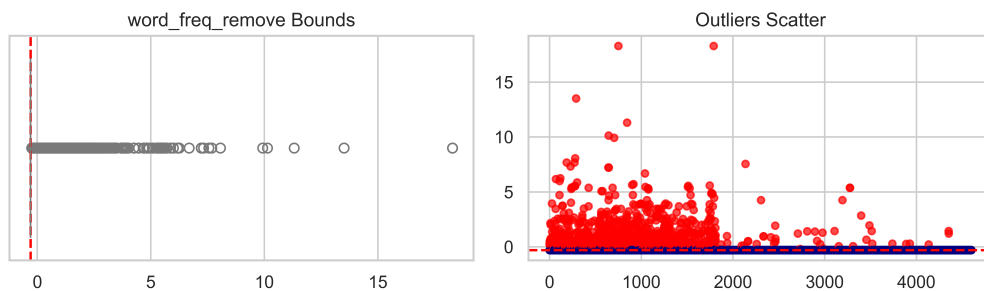


Figure 12: Feature 3: Word Frequency ‘Remove’ – Outliers

Inference: The handful of high-frequency outliers line up with automated bulk-email templates that repeat the opt-out instructions multiple times.

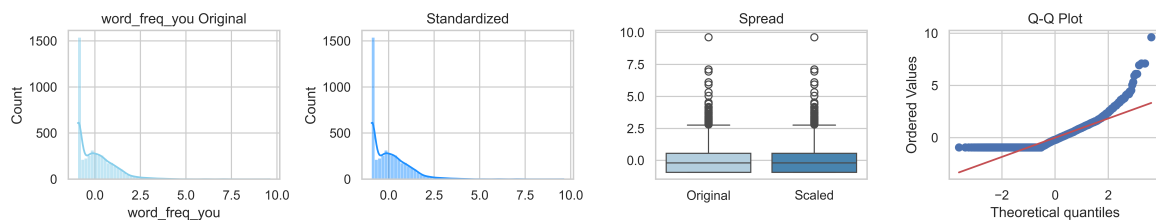


Figure 13: Feature 4: Word Frequency ‘You’ – Normality Distribution

Inference: “You” is used heavily in direct promotional writing (“you’ve won”, “you qualify”), so its distribution is shifted noticeably higher for spam.

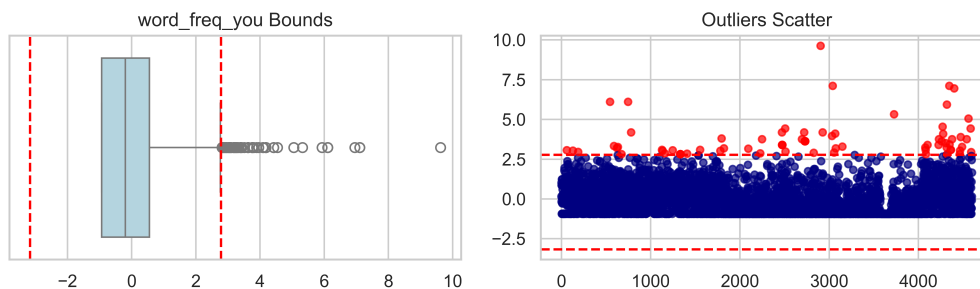


Figure 14: Feature 4: Word Frequency ‘You’ – Outliers

Inference: The outliers are mostly short, aggressively worded promotional emails that repeat “you” several times per sentence.

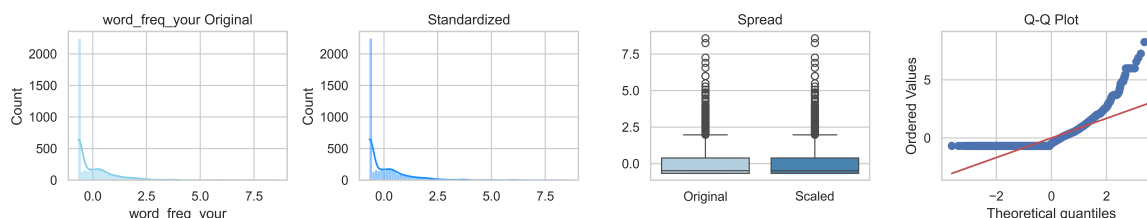


Figure 15: Feature 5: Word Frequency ‘Your’ – Normality Distribution

Inference: As the top-ranked feature from the ANOVA test, “your” shows the clearest separation of any single word – personal and work correspondence rarely uses it as often as promotional spam does.

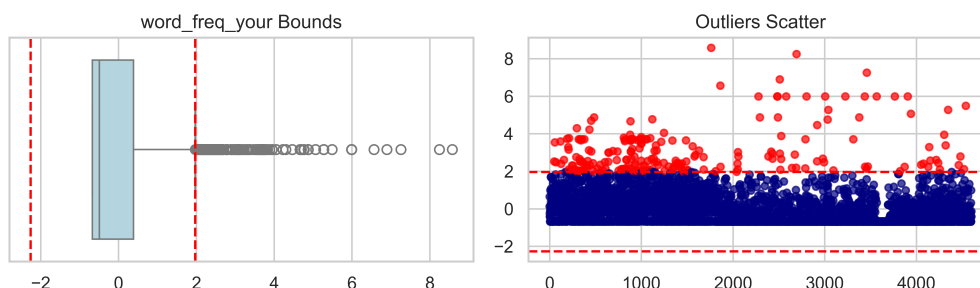


Figure 16: Feature 5: Word Frequency ‘Your’ – Outliers

Inference: The outlying points mostly belong to urgent call-to-action spam (“your account will be suspended”, “claim your prize”) that repeats the word for emphasis.

10 Discussion

Comparing Logistic Regression and SVM on the Spambase dataset shows a smaller gap between a simple linear model and a more flexible non-linear one than expected. Logistic

Regression sets a strong baseline on its own – 92.94% accuracy, 92.09% precision and an F1-score of 90.93%. Since it optimizes a single flat hyperplane through the 57-dimensional feature space, it trains almost instantly and its weights can be read directly to see which features push a prediction toward spam or ham.

The RBF-kernel SVM performs almost identically on the held-out test set (92.73% accuracy), and edges ahead on precision (92.77%). By implicitly mapping the data into a higher-dimensional space through $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma\|\mathbf{x}_i - \mathbf{x}_j\|^2)$, it can capture non-linear interactions between features that a straight line cannot. What stood out more, though, was the 5-fold cross-validation result: SVM was ahead of Logistic Regression in every single fold, averaging 93.40% versus 92.39%. That’s a more convincing signal than the single train/test split, and it suggests SVM’s margin-based boundary generalizes a little more reliably across different samples of the data, even though the two models land on very similar numbers when measured just once. Regularization played a visible role too – the grid search picked L1 for Logistic Regression, which drives some feature weights to exactly zero and effectively performs feature selection, while for SVM the chosen $C = 1$ favoured a wider, less aggressive margin over one that tried to fit every training point perfectly. Standardizing the features beforehand was essential for the SVM side of things: without it, the very large capital-run-length values would have dominated the margin calculations and pushed the model toward a distorted, less useful boundary.

Overall, the bias-variance trade-off between the two models is fairly clear. Logistic Regression, with its single linear boundary, sits on the higher-bias, lower-variance end – fast, stable, and easy to interpret, but limited to boundaries a straight line can express. SVM with an RBF kernel sits a bit further toward the lower-bias, higher-variance end – more capable of fitting complex patterns, but slower to train and its predictions are much harder to explain in plain terms.

11 Conclusion

Both Logistic Regression and Support Vector Machines were trained and tuned on the Spambase dataset to separate spam from ham. Logistic Regression turned out to be a fast, interpretable, and genuinely competitive baseline, reaching 92.94% test accuracy in a fraction of a second. The RBF-kernel SVM matched it closely on the test set and pulled ahead in cross-validation, at the cost of longer training time and a much less transparent decision process. Given how close the two models end up, the choice between them in practice would come down to the priorities of the deployment: Logistic Regression if speed and interpretability matter (for example, needing to explain why an email was flagged), or SVM if squeezing out that last bit of cross-validated accuracy is the priority and training time isn’t a concern.

12 References

- [1] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [2] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.

- [3] UCI Machine Learning Repository, "Spambase Data Set," 1999. [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/spambase>.